VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

HA DU TUAN – 524H0134
NGUYEN DUY KHANG – 524H0017

MIDTERM ESSAY

WEB PROGRAMMING AND APPLICATIONS

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

HA DU TUAN – 524H0134
NGUYEN DUY KHANG – 524H0017

MIDTERM ESSAY

WEB PROGRAMMING AND APPLICATIONS

Advised by
Dr. Le Van Vang

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We would like to express our sincerest gratitude to Professor Le Van Vang for his excellent guidance and dedicated support throughout our studies. Throughout this journey, he has been a tremendous source of motivation and inspiration. His detailed feedback and insightful suggestions have helped us overcome many technical challenges and significantly improve the quality of our work. He patiently guided us through complex concepts and encouraged us to think creatively and innovatively. Beyond the classroom, he taught us invaluable lessons about professionalism, responsibility, and teamwork. His dedication to teaching and passion for technology have left a deep impression on us. We are truly grateful for his time, patience, and unwavering belief in our abilities. It is an honor to have learned from such a knowledgeable and caring teacher. Thank you for everything.

Ho Chi Minh city, 15^{th} *May 2026.*

Author

Tuan

Ha Du Tuan

Khang

Nguyen Duy Khang

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by Mr. Trinh Hung Cuong; The content research and results contained herein are central and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the main author from different sources, which are clearly stated in the reference section.

In addition, the project also uses some comments, assessments as well as data of other authors, other organizations with citations and annotated sources.

If something wrong happens, We'll take full responsibility for the content of my project. Ton Duc Thang University is not related to the infringing rights, the copyrights that We give during the implementation process (if any).

Ho Chi Minh city, 15th *May 2026.*

Author

Tuan

Ha Du Tuan

Khang

Nguyen Duy Khang

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION

1.1 Overview of WebAssembly

WebAssembly (Wasm) is an open standard binary bytecode format designed to run inside web browsers with performance approaching that of native software. An important point to understand from the outset: Wasm is not a programming language for developers to write directly. It is a compilation target, meaning developers write code in C, C++, or Rust, then use tooling to compile that code into the Wasm format, which the browser then executes.

According to the official W3C definition (2019), Wasm is a low-level assembly language for a stack-based virtual machine, with a compact binary format, near-native load and execution times, and a safe sandboxed memory model.

As an analogy, consider a powerful image processing application written in C++. Previously, web browsers could not run it. Wasm acts as a bridge, converting that software into a form the browser understands and can execute at nearly native speed.

1.2 Motivation and Scope

JavaScript was designed for user interface interactions, not heavy computation. When tasked with computationally intensive operations, JavaScript suffers from dynamic typing overhead (type checks at runtime), unstable JIT compilation (the browser may deoptimize unexpectedly), and garbage collection pauses. WebAssembly was created to address these limitations by providing a deterministic, high-performance execution environment alongside JavaScript, not as a replacement for it.

This report examines the theoretical foundations of WebAssembly, compares it with JavaScript and legacy alternatives, and illustrates its practical applications in industry.

CHAPTER 2. THEORETICAL BASIS

2.1 Historical Background

WebAssembly did not emerge suddenly; it is the result of a long iterative process through many experiments and failures:

1. 1995 — JavaScript introduced: The only language running natively in browsers. Designed for UI interactions, not heavy computation.
2. 2010 — Google experiments with Native Client (NaCl): Attempted to run native code in the browser, but failed due to architecture-specific restrictions and lack of portability.
3. 2013 — Mozilla introduces asm.js: An optimized subset of JavaScript, considered the direct predecessor of Wasm. Still slow because browsers had to parse full JavaScript text.
4. 2015 — Wasm announced: For the first time, the four major browser vendors (Google, Mozilla, Microsoft, and Apple) collaborated on a single open standard.
5. 2017 — Wasm officially launched: The MVP (Minimum Viable Product) was integrated into all four major browsers: Chrome, Firefox, Edge, and Safari.
6. 2019 — W3C recognizes Wasm as a Web Standard: Wasm becomes the fourth web standard alongside HTML, CSS, and JavaScript.
7. 2022 onwards — Wasm beyond the browser: With WASI (WebAssembly System Interface), Wasm begins deployment on servers, in cloud environments, and beyond the browser context.

2.2 Technical Architecture

2.2.1 Stack-Based Virtual Machine

Wasm runs on a virtual machine inside the browser, operating in a stack-based manner where all computations are performed by pushing and popping values on a

virtual stack. Wasm supports exactly four primitive data types: i32, i64, f32, and f64. This intentional minimalism allows browsers to compile Wasm to real machine code extremely fast.

2.2.2 Linear Memory Model

Wasm does not use shared memory in the conventional sense. Instead, it has a linear memory: a contiguous byte array created from JavaScript memory. Both JavaScript and Wasm can read and write to this shared region, which is the primary mechanism for data exchange between the two environments. There is no automatic garbage collection for this region; the developer manages memory manually (or delegates to the source language runtime such as C++ malloc/free).

2.2.3 Module, Instance, and Import/Export

A Wasm Module is an independent deployment unit, similar to a complete software package. Each module has four key components: Imports (functions or memory that Wasm needs JavaScript to provide, such as console output or random number generation); Functions (the bytecode implementing computation logic); Exports (functions Wasm exposes for JavaScript to call, such as image processing or encryption routines); and Table (a reference table for function pointers when compiling from C++).

2.2.4 Sandbox Security Model

All Wasm code runs in a fully isolated sandbox. Wasm cannot access the file system, cannot invoke system calls (syscalls), can only read/write within its own Linear Memory, and all interactions with browser APIs must go through a controlled JavaScript bridge. If Wasm code attempts to read memory outside its allowed region, execution is immediately terminated, preventing C-style buffer overflow vulnerabilities.

2.3 Compile Toolchain

To produce a .wasm file, developers use a compile toolchain. Two primary paths exist:

Emscripten (for C/C++): The most widely used toolchain, developed by Mozilla. It uses the Clang/LLVM compiler to convert C/C++ code to WebAssembly via the pipeline: C/C++ source code → LLVM IR (intermediate representation) → binary .wasm file plus a JavaScript glue file for browser integration. Emscripten is ideal for porting well-known C++ libraries such as OpenCV to the web.

wasm-pack (for Rust): A toolchain for the Rust language that automatically produces the .wasm file, a JavaScript connector, and TypeScript definitions. Compared to Emscripten, wasm-pack produces smaller output and integrates with JavaScript more easily through the wasm-bindgen code generator. Rust’s borrow checker enforces memory safety at compile time, eliminating entire classes of memory errors that are possible in C++.

CHAPTER 3. DESIGNING AND IMPLEMENTING

3.1 System Architecture — UML Diagrams

The system design for a WebAssembly Image Processing application is illustrated through four UML diagrams below. Figure 3.1 shows the Use Case Diagram, outlining user interactions with the Wasm Engine and JS Engine. Figure 3.2 shows the Class Diagram, depicting the relationships among key software components. Figure 3.3 shows the Sequence Diagram, tracing the runtime flow from image upload through Wasm processing to result rendering. Figure 3.4 shows the Entity-Relationship Diagram (ERD), defining the data model for persistence.

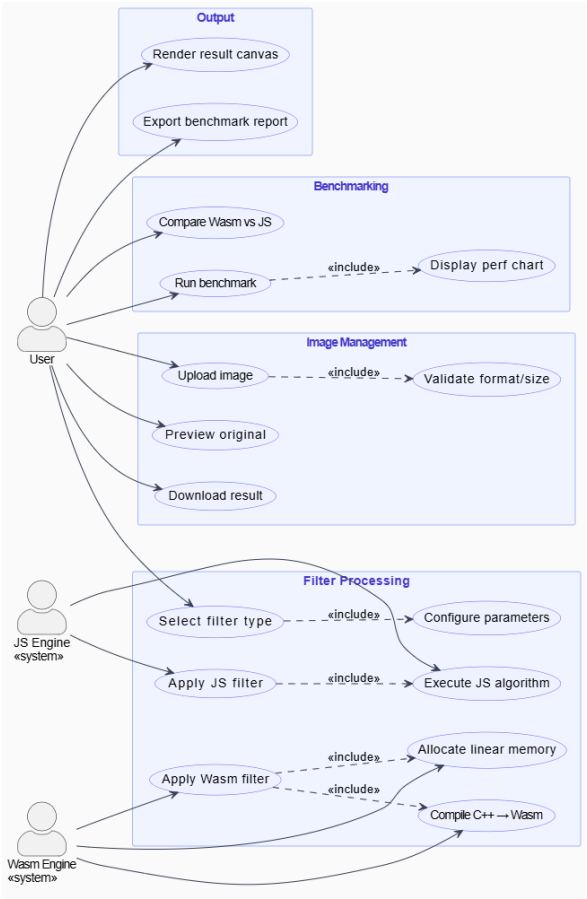

Figure 3.1: Use Case Diagram — WebAssembly Image Processing Application

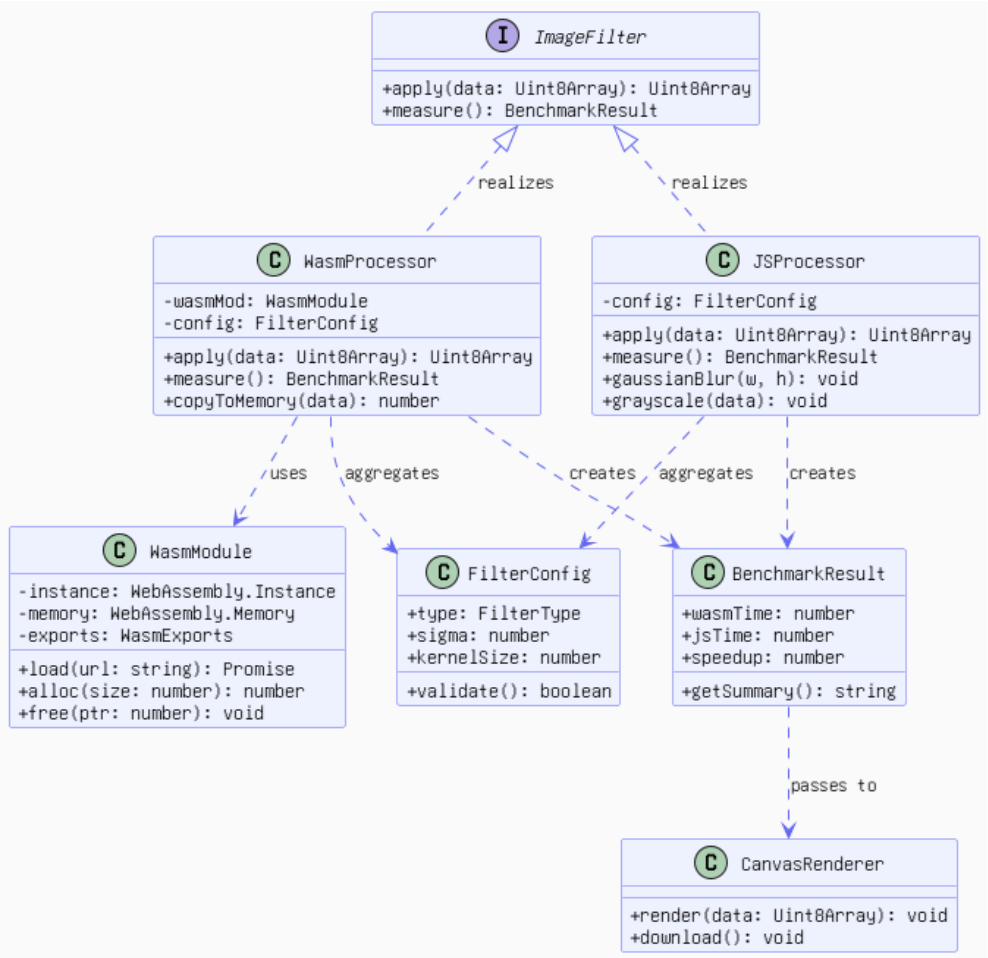

Figure 3.2: Class Diagram — System Component Relationships

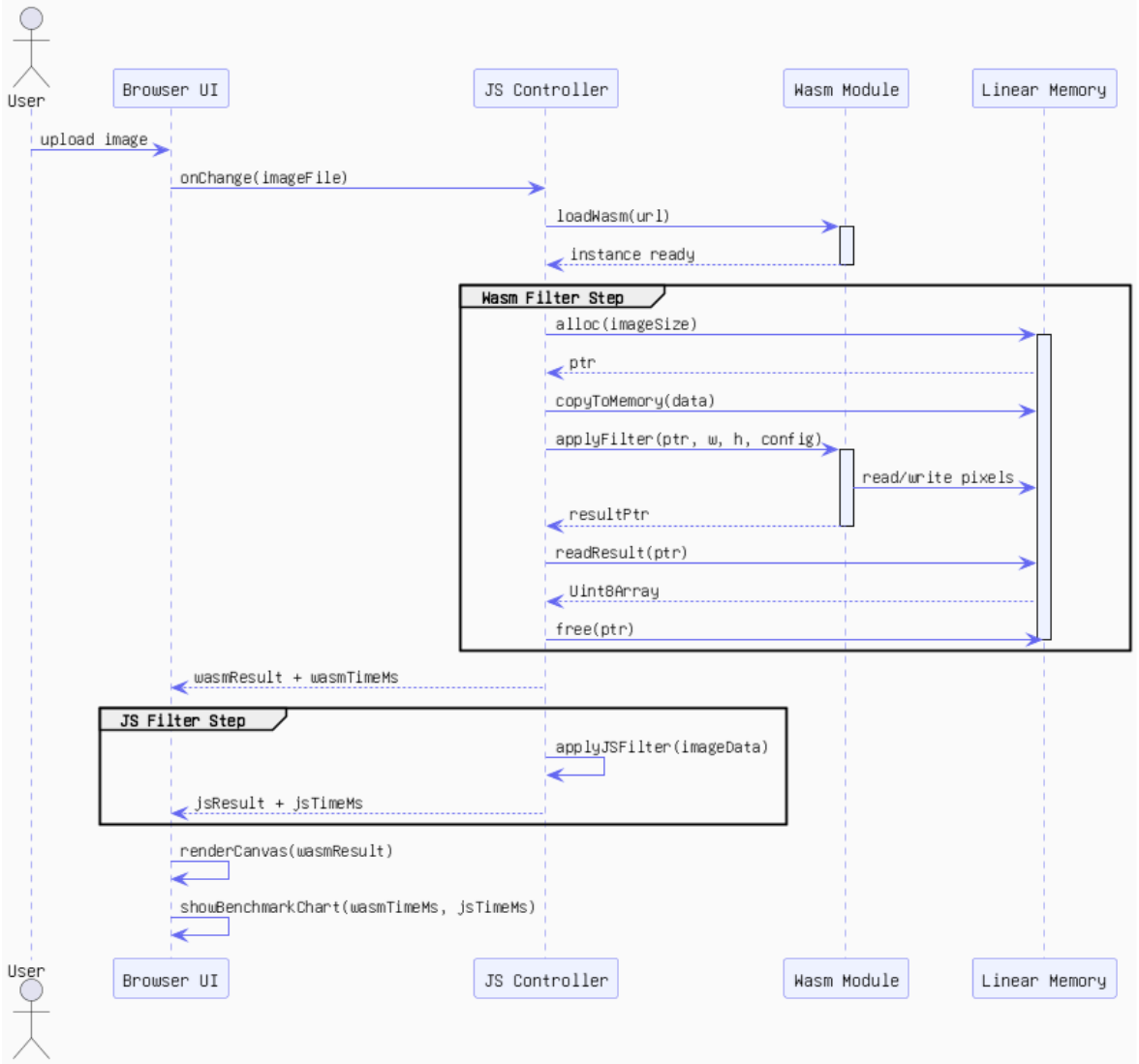

Figure 3.3: Sequence Diagram — Wasm vs JS Filter Execution Flow

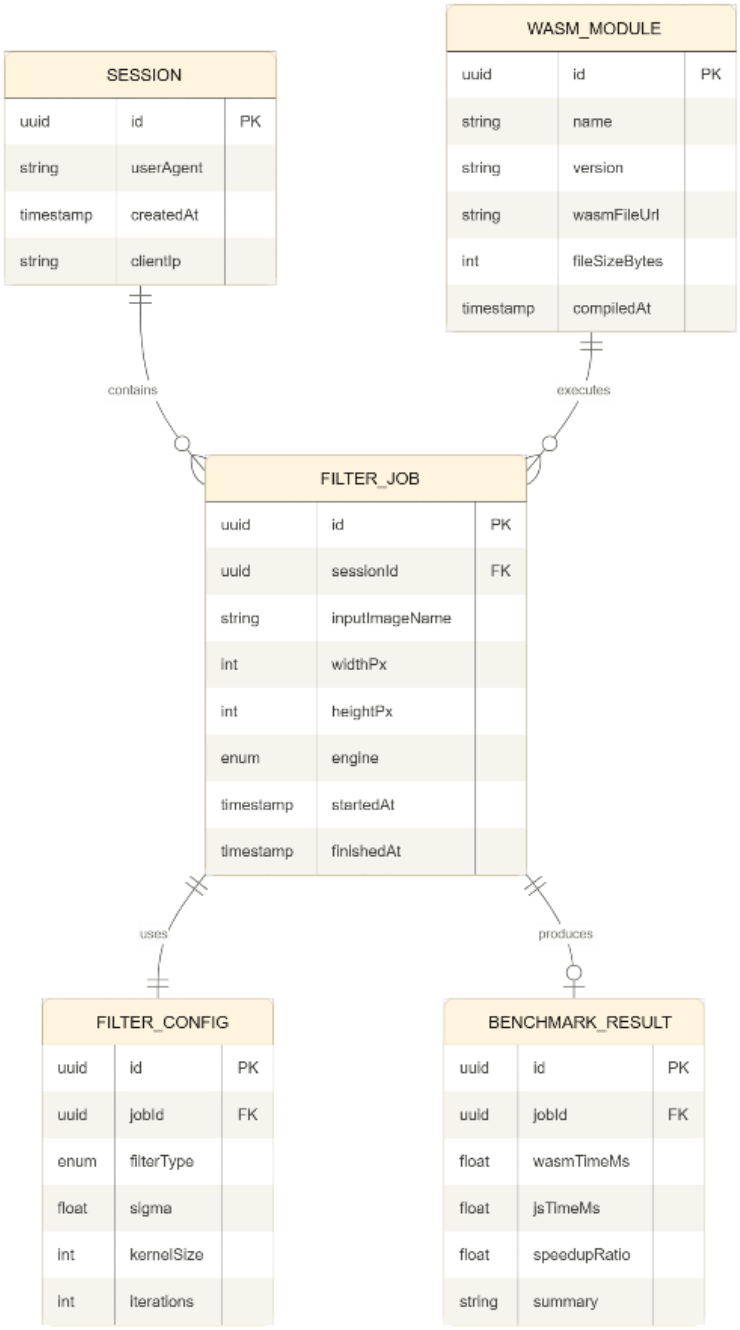

Figure 3.4: Entity-Relationship Diagram (ERD) — Data Model

3.2 Key Implementation Considerations

The Wasm filter pipeline follows these steps: (1) The user uploads an image via the Browser UI; (2) the JS Controller loads the Wasm module and allocates linear memory; (3) pixel data is written into linear memory via writePixels(); (4) the Wasm module executes the Gaussian Blur algorithm by reading and writing pixels

in-place; (5) the JS Controller reads the result back and renders it on the HTML canvas.

For the JavaScript filter comparison path, the applyJSFilter() function executes an equivalent Gaussian Blur using pure JavaScript, capturing execution time for benchmarking purposes. Both execution times are recorded in the BENCHMARK_RESULT table as defined in the ERD.

CHAPTER 4. RESULT

4.1 Performance Comparison

According to benchmark studies from Mozilla, the Google Chrome team, and academic research (Jangda et al., 2019), Wasm outperforms JavaScript by approximately 6 to 7 times for computationally intensive tasks such as Gaussian Blur image processing, SHA-256 hashing, Fast Fourier Transform (FFT), and matrix multiplication. For DOM manipulation tasks (modifying the web page interface), Wasm provides no advantage because it must pass through the JavaScript bridge.

4.2 Advantages and Disadvantages

Advantages of WebAssembly include: near-native performance (2 to 10 times faster than JavaScript for CPU-bound tasks); portability (the same .wasm file runs on all modern browsers and operating systems); code reuse (enabling well-known libraries such as OpenCV, FFmpeg, and SQLite to run on the web); sandbox security equivalent to JavaScript; language independence (C, C++, Rust, Go, Swift, and Kotlin can all compile to Wasm); and seamless JavaScript interoperability.

Disadvantages include: no direct DOM access (all UI interactions must go through the JavaScript bridge, adding overhead); difficult debugging (stack traces display bytecode addresses rather than source code lines); large output files (Emscripten output is typically 300 KB to 2 MB due to the POSIX emulation layer); absence of a built-in garbage collector; startup latency (loading, validating, and compiling the .wasm file takes 100 to 500 ms on first load); and data transfer overhead when copying data between JavaScript and Wasm through Linear Memory.

CHAPTER 5. CONCLUSION

5.1 Summary

WebAssembly represents a significant leap in web platform capabilities. By providing a deterministic, sandboxed, near-native execution environment, it bridges the gap between native application performance and the portability of the web. It complements JavaScript rather than replacing it, handling the computationally intensive operations that JavaScript handles poorly, while JavaScript continues to manage DOM interactions, event handling, and network communication.

Practical adoption at scale validates the technology: Figma uses Wasm to run its C++ graphics engine in the browser (3x faster than JavaScript); Google Earth Web renders 3D global maps; Autodesk ported over one million lines of C++ code accumulated over 35 years to the web without a rewrite; Adobe Photoshop Web processes images in real time; and TensorFlow.js uses Wasm as an inference backend approximately 10 times faster than JavaScript for machine learning tasks.

5.2 Potential and Development Directions

With WASI (WebAssembly System Interface), Wasm extends beyond the browser to servers, edge computing, and cloud-native environments. Future developments include garbage collection support (enabling languages such as Python and Ruby to target Wasm more efficiently), interface types for richer JavaScript/Wasm interoperability, multi-threading via SharedArrayBuffer, and SIMD instructions for vectorized computation. WebAssembly is poised to become a universal compilation target for safe, portable, high-performance computing across all platforms.

Link GitHub: https://github.com/khanglhlt-hash/Web_Program_and_Application_Mid_Term

Link Video Demo: https://drive.google.com/file/d/1Wn5-OAVNUikxoWaQZhCOSQjv2o7Up3zd/view?usp=drive_link

REFERENCES

Akarsh Shrivas, Aniket Pawar, Pratham Mishra, Satish Chadokar. (2023). *E-commerce Website using MERN Stack- IJIRMPS Volume 11, Issue 3.* IJRASET.

Shubham, K. (2021, January 23). *Build an E-Commerce website with MERN Stack* . Retrieved from dev.to: https://dev.to/shubham1710/build-an-e-commerce-website-with-mern-stack-part-1-setting-up-the-project-1l2d